

Module 2: Cheat Sheet: Spring Boot for Database Applications

Package/Method	Description	Code Example
Add the MySQL JDBC dependency to the pom.xml file	The mysql-connector-java dependency is added to the pom.xml file with a scope of runtime. This dependency provides the Java API that enables Java applications to interact with MySQL databases using JDBC.	<pre> <dependencies> <!-- MySQL JDBC Driver --> <dependency> <groupId>mysql</groupId> <artifactId>mysql-connector-java</artifactId> <scope>runtime</scope> </dependency> </dependencies> </pre>
Define the database connection properties	Define the database connection properties in the application.properties file located in src/main/resources. The application.properties file centralizes your application configuration. spring.datasource.url specifies the database URL. spring.datasource.username and spring.datasource.password define the credentials for database access.	<pre> # Database connection properties only spring.datasource.url=jdbc:mysql://localhost:3306/bookstore spring.datasource.username=root spring.datasource.password=password spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver </pre>

Customize and activate profiles for different environments	Spring Boot supports multiple environment profiles (dev, prod, etc.) through application-{profile}.properties. The spring.profiles.active setting determines which profile is currently in use, allowing different database configurations based on the execution environment.	<pre># application-dev.properties spring.datasource.url=jdbc:mysql://localhost:3306/bookstore_dev # application-prod.properties spring.datasource.url=jdbc:mysql://prod-server:3306/bookstore # Profile activation spring.profiles.active=dev</pre>
Set up logging	Spring Boot logging is configured using logging.level properties. Log files can be stored using logging.file.name.	<pre>logging.level.org.springframework=INFO logging.level.com.yourapp=DEBUG logging.file.name=logs/app.log</pre>
Securing sensitive data	To secure credentials, environment variables (DB_USERNAME, DB_PASSWORD) should be used instead of hardcoding values in the application.properties file.	<pre>spring.datasource.username=\${DB_USERNAME} spring.datasource.password=\${DB_PASSWORD} export DB_USERNAME=root export DB_PASSWORD=securepassword</pre>

Initialize the schema	The schema.sql file defines the structure of tables in the database. When Spring Boot starts, it executes this script to create tables if they do not exist. The CREATE TABLE statement defines column names, data types, and constraints.	<pre>CREATE TABLE Books (id INT AUTO_INCREMENT PRIMARY KEY, title VARCHAR(255), author VARCHAR(255), price DOUBLE);</pre>
Seed the database with initial data	data.sql seeds the database with initial data.	<pre>INSERT INTO Books (title, author, price) VALUES ('Spring Boot Essentials', 'Jane Doe', 29.99); Add some more values INSERT INTO Books (title, author, price) VALUES ('Spring Boot Essentials', 'Jane Doe', 29.99), ('Mastering Hibernate', 'John Smith', 39.99), ('Java Persistence API Guide', 'Emily Davis', 25.50);</pre>

List the databases	The code creates a statement with an existing connection to a database. The statement is then used to execute the query \"Show databases\". The result is stored in resultSet and then printed out to the console using a while loop. This is a simple way to ensure the code can connect to the database.	<pre>try (Statement statement = connection.createStatement()); ResultSet resultSet = statement.executeQuery("SHOW DATABASES") { System.out.println("Available Databases:"); while (resultSet.next()) { System.out.println(resultSet.getString(1)); } } catch (SQLException e) { System.err.println("Error retrieving databases: " + e.getMessage()); e.printStackTrace(); } }</pre>
Connect to the database and list the tables	To list the tables, first create a connection to the database using the database URL, username, and password. The credentials should be stored securely and not hardcoded as shown here. We are doing this for simplicity's sake. The code creates a statement with the query "show tables" and the result is stored in the variable resultSet. A while statement then loops over it and prints the tables to the console.	<pre>try (Connection connection = DriverManager.getConnection("jdbc:mysql://localhost:3306/bookstore", "root", "password"); Statement statement = connection.createStatement(); ResultSet resultSet = statement.executeQuery("SHOW TABLES")) { System.out.println("Tables in the database:"); while (resultSet.next()) { System.out.println(resultSet.getString(1)); } } catch (SQLException e) { System.err.println("Error: " + e.getMessage()); e.printStackTrace(); } }</pre>

Data health checks: Add the Spring Actuator dependency to your pom.xml.	Adding the spring-boot-starter-actuator dependency enables application monitoring, including database health checks.	<pre> <dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-actuator</artifactId> </dependency> </pre>
Data health check: Enable the database health endpoint	The management.health.db.enabled setting allows Spring Boot's Actuator to check the status of the database connection via a REST endpoint (/actuator/health/db).	<pre> management.endpoint.health.show-details=always management.health.db.enabled=true Output: "status": "UP", "components": { "db": { "status": "UP", "details": { "database": "MySQL", "validationQuery": "isValid()" } } } } </pre>

		}
Setting up JPA in Spring Boot: Add dependencies to enable JPA in SpringBoot	The spring-boot-starter-data-jpa dependency enables JPA (Java Persistence API), allowing ORM (Object-Relational Mapping) with MySQL databases.	<pre><dependencies> <dependency> <groupId>org.springframework.boot</groupId> <artifactId>spring-boot-starter-data-jpa</artifactId> </dependency> <dependency> <groupId>mysql</groupId> <artifactId>mysql-connector-java</artifactId> <scope>runtime</scope> </dependency> </dependencies></pre>
	Set the database and JPA configuration in src/main/resources/application.properties.	<pre>spring.datasource.url=jdbc:mysql://localhost:3306/bookstore spring.datasource.username=root</pre>

Setting up JPA in Spring Boot: Configure application.properties	spring.jpa.hibernate.ddl-auto automatically manages the database schema, and the update part updates the schema based on your JPA entities. So, if your database is missing some tables but your JPA has the right classes, those missing tables will be auto-created. Other options for this setting would be create, validate, or none. spring.jpa.show-sql logs the generated SQL queries for debugging purposes.	<pre>spring.datasource.password=password spring.jpa.hibernate.ddl-auto=update spring.jpa.show-sql=true</pre>
Setting up JPA in Spring Boot: Define an entity	An entity is a Java class mapped to a database table. In this code, @Entity is an annotation that marks the class as a JPA entity mapped to a database table. @Id is an annotation that denotes the primary key of the entity. @GeneratedValue is an annotation that specifies how the primary key is generated, such as AUTO or IDENTITY.	<pre>@Entity public class Book { @Id @GeneratedValue(strategy = GenerationType.IDENTITY) private Long id; private String title; private String author; private double price; // Getters and Setters }</pre>
	In this code, JpaRepository is a built-in	<pre>@Repository public interface BookRepository extends JpaRepository <Book,Long> { }</pre>

Setting up JPA in Spring Boot: Create a repository	Spring Data JPA interface that provides methods like save, findAll, and deleteById. Book, Long specifies the entity type, Book, and the type of its primary key, Long	
Using JPA: Bookstore application example	In this code snippet, the 'setTitle' method on the 'book' object sets the title, Similarly, the 'setAuthor' method sets the author, and the 'setPrice' method sets the price. The 'save' method of the bookRepository object saves the Book object to the database.	<pre> @Component public class BookApp implements CommandLineRunner { @Autowired private BookRepository bookRepository; @Override public void run(String...args) { Book book = new Book(); book.setTitle("Spring Boot Essentials"); book.setAuthor("Jane Doe"); book.setPrice(19.99); bookRepository.save(book); System.out.println("Book saved successfully!"); } } </pre>
		<pre> @Entity public class Book { </pre>

Apply Java core annotations to a JPA entity	The NotNull annotation on the title ensures the title cannot be null. The same applies to the author attribute. The Size annotation restricts the title to a length between 2 and 100 characters. The Min annotation ensures the price is at least 1.	<pre> @Id private Long id; @NotNull @Size(min = 2, max = 100) private String title; @NotNull private String author; @Min(1) private double price; // Getters and Setters } </pre>
Spring-specific validation options: Example	First, import the required packages. The validator class is the core of Spring's validation framework. It allows us to implement custom validation logic for any object. The errors class is used to store and retrieve validation errors during validation processing. The ValidationUtils class is used to automatically check for empty fields and adds errors if a field is blank or null. The Implements Validator class implements the Validator interface, which requires defining two methods: supports and validate.	<pre> import org.springframework.validation.Errors; import org.springframework.validation.ValidationUtils; import org.springframework.validation.Validator; public class BookValidator implements Validator { @Override public boolean supports(Class<?> clazz) { return Book.class.equals(clazz); // Ensures this validator applies only to Book objects. } @Override public void validate(Object target, Errors errors) { Book book = (Book) target; // Ensures the price is greater than 0 if (book.getPrice() < 1) { errors.rejectValue("price", "price.invalid", "Price must be greater than 0"); } // Ensures the title is not empty ValidationUtils.rejectIfEmptyOrWhitespace(errors, "title", "title.empty", "Title cannot be empty"); } } </pre>

	The Supports method takes in a class type as an input parameter and returns "true" only if the given class is Book.class. The Validate method contains the validation rules. It first converts the target object into a Book and then checks if the price is less than 1. If true, it returns an error message stating that the price has to be greater than zero. The ValidationUtils class is used to check if the title is empty or contains only spaces. This ensures the caller does not send an empty title or blank string.	
Use the Validator in Spring Controllers: Validate user input dynamically	The Validator interface can be integrated into Spring controllers to validate user input dynamically. The Controller annotation marks this class as a Spring MVC controller, allowing it to handle HTTP requests. The bookValidator object created previously manually validates Book objects before processing them. It calls the validate method from BookValidator to check if the Book object meets the defined constraints. This controller ensures that any book added through the addBook endpoint is validated before processing, preventing invalid data from being stored or processed. result.hasErrors checks if validation errors exist. If true, it returns "errorPage", indicating validation failed. If false, it proceeds with normal processing and returns "successPage".	<pre>@Controller public class BookController { private final BookValidator bookValidator = new BookValidator(); @PostMapping("/addBook") public String addBook(@ModelAttribute Book book, BindingResult result) { bookValidator.validate(book, result); // Validate the book object. if (result.hasErrors()) { return "errorPage"; // Return an error page if validation fails. } // Process the book object if validation passes. return "successPage"; } }</pre>
		<pre>Book book = new Book(); // The target object.</pre>

The DataBinder class	New DataBinder creates a DataBinder instance for the book object. dataBinder.setValidator attaches the custom BookValidator to the DataBinder. This is the validator you created previously. dataBinder.bind simulates binding input data, such as input from an HTTP request, to the book object. dataBinder.validate executes runs the validation logic defined in the BookValidator. BindingResult.getAllErrors retrieves all validation errors for further processing.	<pre> DataBinder dataBinder = new DataBinder(book); // Set the custom validator. dataBinder.setValidator(new BookValidator()); // Simulate binding input data. dataBinder.bind(new MutablePropertyValues(Map.of("title", "", "price", 0))); // Validate the object. dataBinder.validate(); // Retrieve validation errors. BindingResult result = dataBinder.getBindingResult(); if (result.hasErrors()) { result.getAllErrors().forEach(System.out::println); } </pre>
Handle validation in REST API: Global exception handler	The ControllerAdvice annotation designates the class as a centralized exception handler. The ExceptionHandler annotation captures specific exceptions. In this case, it captures MethodArgumentNotValidException. MethodArgumentNotValidException is thrown when a request body fails validation. Whenever validation fails, this method gets called. The getFieldErrors method retrieves all validation errors from the exception.	<pre> @ControllerAdvice public class GlobalExceptionHandler { @ExceptionHandler(MethodArgumentNotValidException.class) public ResponseEntity<Map<String, String>> handleValidationExceptions(MethodArgumentNotValidException ex) { Map<String, String> errors = new HashMap<>(); ex.getBindingResult().getFieldErrors().forEach(error -> errors.put(error.getField(), error.getDefaultMessage())); return new ResponseEntity<>(errors, HttpStatus.BAD_REQUEST); } } </pre>

	The forEach statement loops through each validation error, extracts the field name and the error message, and adds them to the errors map. HttpStatus.BAD_REQUEST returns a 400 Bad Request status code for invalid requests.	
Standardize API responses	The ApiResponse class includes three key fields. The status field indicates whether the request was successful. The message field provides a short description of the outcome. The data field contains the actual response payload.	<pre>public class ApiResponse<T> { private String status; private String message; private T data; public ApiResponse(String status, String message, T data) { this.status = status; this.message = message; this.data = data; } }</pre>
Modify the controller methods to use this	The controller method modifies API responses to follow this structure. If a book is found, the API returns a successful response. If not, it returns a structured error response.	<pre>@GetMapping("/{id}") public ResponseEntity<ApiResponse<Book>> getBookById(@PathVariable Long id) { return bookRepository.findById(id) .map(book -> ResponseEntity.ok(new ApiResponse<>("success", "Book found", book))) .orElseGet(() -> ResponseEntity.status(HttpStatus.NOT_FOUND) .body(new ApiResponse<>("error", "Book not found", null))); }</pre>

structured response	This approach ensures consistency and improves front-end handling.	
Return proper status codes	This example shows how to return a 201 status when adding a new book. The <code>ResponseEntity.status(HttpStatus.CREATED)</code> method sets the status code, and the <code>ApiResponse</code> object structures the response.	<pre> @PostMapping public ResponseEntity<ApiResponse<Book>> createBook(@Valid @RequestBody Book book) { Book savedBook = bookRepository.save(book); return ResponseEntity.status(HttpStatus.CREATED) .body(new ApiResponse<>("success", "Book created successfully", savedBook)); } </pre>
Retrieve all books using a simple Statement	In this code, the statement query is executed using 'SELECT asterisk FROM Books', and the results are traversed and printed out. As before a 'catch' block captures any exceptions that occur.	<pre> try (Statement statement = connection.createStatement(); ResultSet resultSet = statement.executeQuery("SELECT * FROM Books")) { while (resultSet.next()) { System.out.println("Book ID: " + resultSet.getInt("BookID")); System.out.println("Title: " + resultSet.getString("Title")); System.out.println("Author: " + resultSet.getString("Author")); System.out.println("Price: " + resultSet.getDouble("Price")); } } catch (SQLException e) { System.err.println("Error retrieving data: " + e.getMessage()); e.printStackTrace(); } </pre>

Implement Pagination for Large Datasets	Use pagination to limit the number of records in API responses.	<pre> @GetMapping("/paged") public Page<Book> getPagedBooks(Pageable pageable) { return bookRepository.findAll(pageable); } </pre>
Pagination query example	In this code snippet, the endpoint is 'forward slash paged'. When somebody makes a call to this endpoint, they can pass in certain parameters. In this example, we are passing in page zero, which is the first page, we are passing in a size of 5, so we only want 5 books returned in the response, and we want the results sorted by title in ascending order.	<pre>GET /books/paged?page=0&size=5&sort=title,asc</pre>
Enable SQL query logging	To see SQL queries executed by JPA, add this to application.properties.	<pre>spring.jpa.show-sql=true</pre>

Add logging in controllers	In this code snippet, the import command imports the logger, the 'Logger factory' class is used with 'get Logger' to create the logger instance, and 'logger.info' is used to log the fetched information.	<pre>import org.slf4j.Logger; import org.slf4j.LoggerFactory; @RestController @RequestMapping("/books") public class BookController { private static final Logger logger = LoggerFactory.getLogger(BookController.class); @GetMapping public List<Book> getAllBooks() { logger.info("Fetching all books from database."); return bookRepository.findAll(); } }</pre>
Curl command: POST request	In this code snippet, curl is using the POST method, the 'dash H' flag sets the header, the content type is specified, the dash d flag is used to pass the data, and the URL is the endpoint that is being created.	<pre>curl -X POST -H "Content-Type: application/json" -d '{ "title": "Spring Boot Guide", "author": "John Doe", "price": 20.99, "genre": "Programming", "publicationDate": "2025-03-15" }' http://localhost:8080/books</pre>

Curl command: GET request	The code fetches all books from the database. A GET request is used to retrieve all books from the database. The API returns a list of books in JSON format.	<pre>curl -X GET http://localhost:8080/books</pre>

Author(s)

Upkar Lidder
Sangeeta Srinivasan